

Filebrowser Self-Hosted Deployment Case Study

Docker | Windows 11 | PowerShell | Local Network | Self-Hosted Cloud Storage



Docker

Windows 11

PowerShell

Port 8080

Self-Hosted

Filebrowser v2.63.2

1 PROJECT CONCEPT

This project deploys Filebrowser, an open-source self-hosted web application, as a privacy-respecting replacement for commercial cloud storage. Instead of relying on Google Drive or OneDrive, files are served from a local machine with full ownership, zero recurring cost, and a clean browser-based UI.

Skills Demonstrated

- Docker image pulling, container lifecycle, and bind mount configuration
- Credential recovery through container log inspection
- Principle of least privilege: admin vs. standard user separation
- Service persistence via restart policies
- Web UI administration: user management, global settings, file operations

2 THE STACK

Host OS	Windows 11
Shell	PowerShell 7
Container Runtime	Docker Desktop (latest)
Image	filebrowser/filebrowser:latest (v2.63.2)
Port Mapping	8080 (host) : 80 (container)
Volume Mount	C:\files -> /srv (container)
Restart Policy	--restart unless-stopped

3 DEPLOYMENT WALKTHROUGH

1 Pull & Run the Container

A single docker run command pulled the latest Filebrowser image from Docker Hub and started the container in detached mode. The --restart unless-stopped flag keeps it running after reboots. The bind mount maps C:\files on the host to /srv inside the container so files persist independently of the container.

```
docker run -d --restart unless-stopped -p 8080:80 -v C:\files:/srv filebrowser/filebrowser
```

```
PS C:\Users\keena> docker run -d `
>> --restart unless-stopped `
>> -p 8080:80 `
>> -v C:\files:/srv `
>> filebrowser/filebrowser
Unable to find image 'filebrowser/filebrowser:latest' locally
latest: Pulling from filebrowser/filebrowser
daf933fc4e50: Pull complete
15571b098b3a: Pull complete
ecfee82d31cb: Pull complete
0fe8ebb144ff: Pull complete
6402996961bf: Pull complete
32065fdebaac: Pull complete
6d40c172fb8a: Pull complete
74b709880295: Pull complete
6319a2247213: Pull complete
91e087fc779a: Pull complete
a55215715441: Pull complete
04476609bc75: Pull complete
Digest: sha256:4dce87308b9f9cfbcf8d0a284fc9565d2b515530a6bae2d920b388
161e093f26
Status: Downloaded newer image for filebrowser/filebrowser:latest
a75a2d4b18a05e5ab3c066be9531e40928c90a18a9592c320a2da972eae6ebcf
```

Fig 1. PowerShell output -- Docker pulling all image layers and confirming the download digest

2 First Login Attempt & Credential Discovery

Navigating to 127.0.0.1:8080 presented the login screen. The default admin:admin credentials failed. Rather than guessing, container logs were inspected to find the auto-generated password created during "quick setup."

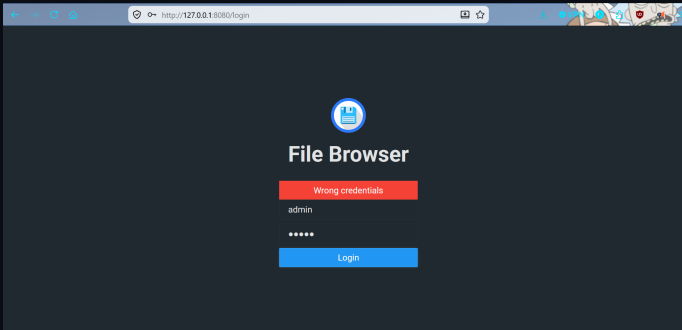


Fig 2. Login screen showing "Wrong credentials" after trying admin:

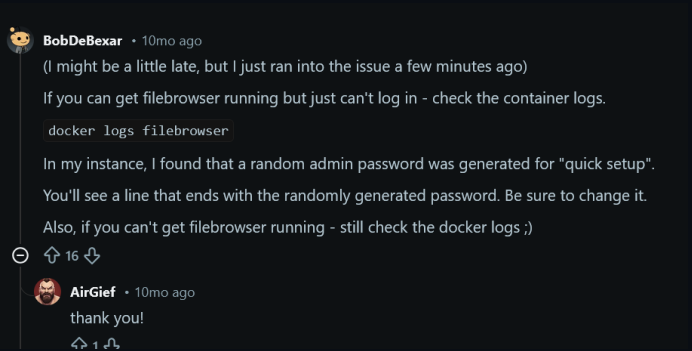


Fig 3. Reddit post confirming the fix: `run docker logs <container_name>`

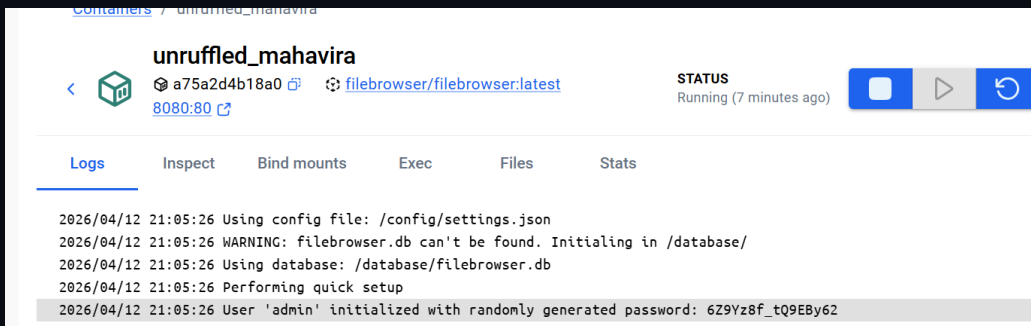


Fig 4. Docker Desktop logs revealing the randomly generated admin password used for first access

3 Change Admin Password

After logging in with the auto-generated password, the admin password was immediately changed to a strong custom passphrase via User Management. Filebrowser requires confirming the current password before saving, preventing silent credential changes.

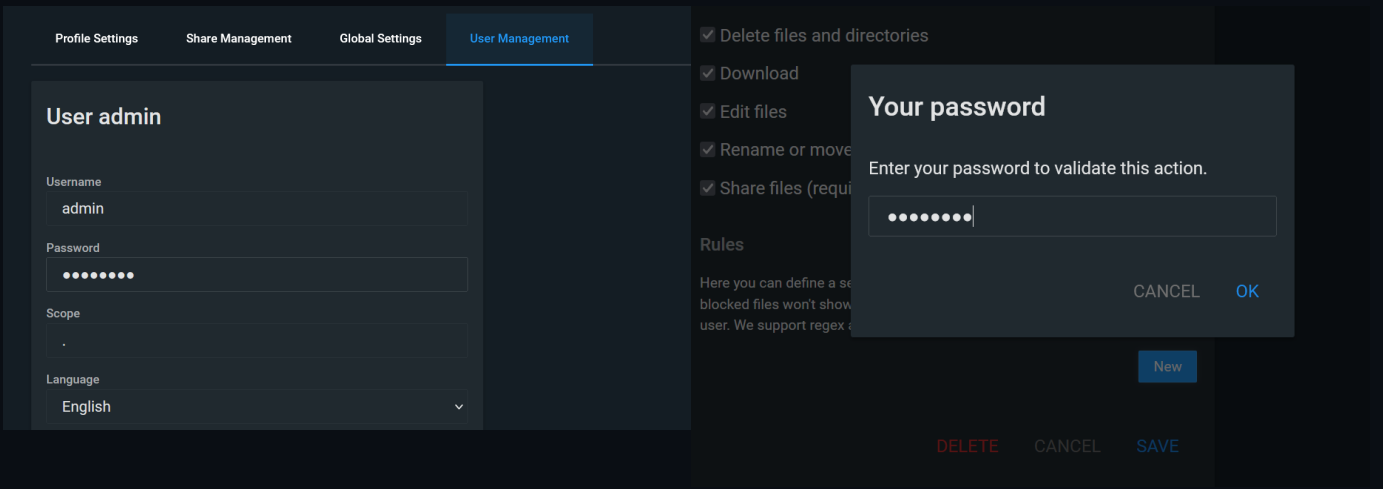


Fig 5. User Management panel with new password entered

Fig 6. Confirmation modal requiring current password to validate the change

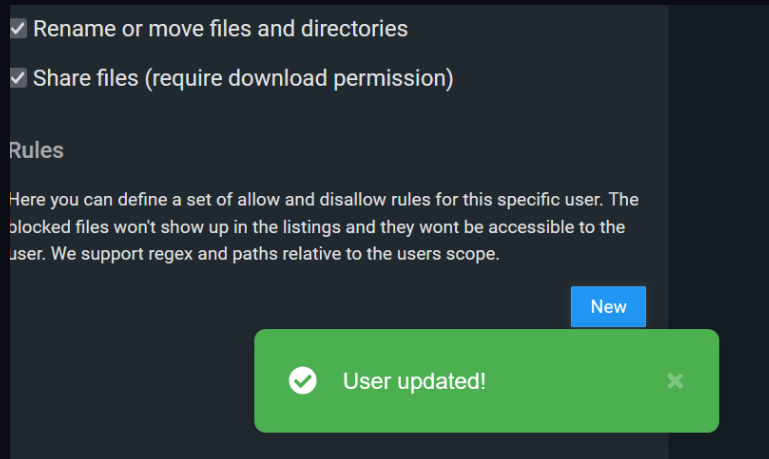


Fig 7. Success toast confirming the admin password was updated

4 Enable User Signup & Create Standard Account

To follow the Principle of Least Privilege, a non-admin account was created for day-to-day file operations. User signup was enabled in Global Settings so accounts can be self-registered without admin involvement each time.

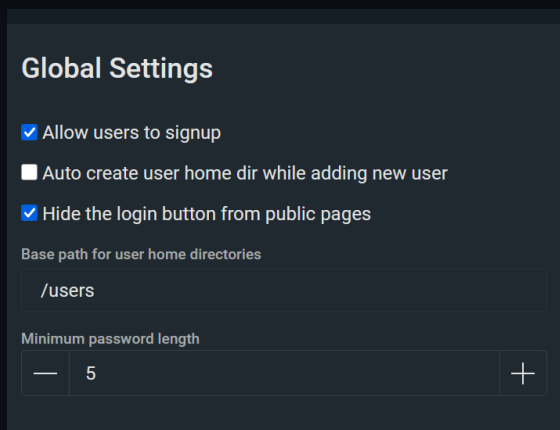


Fig 8. Global Settings -- "Allow users to signup" enabled

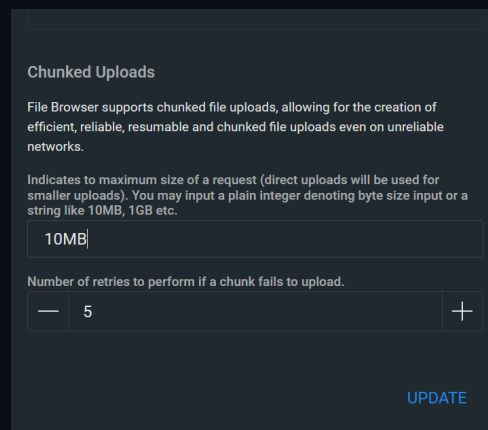


Fig 9. Chunked uploads config (10MB threshold, 5 retries) -- scroll down then UPDATE

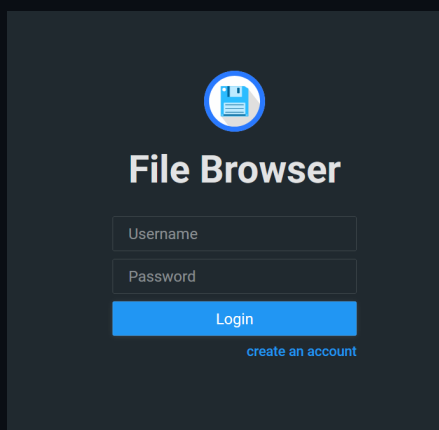


Fig 10. Login page now shows "create an account" link after enabling

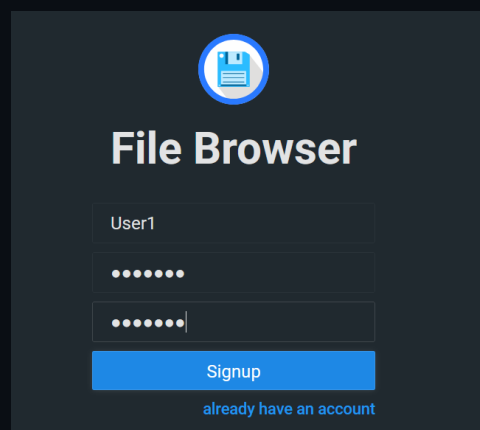


Fig 11. Signup form -- creating User1 with username + password confirmation

5 Access Dashboard & Verify File Operations

User1 logs in and lands on a clean file browser dashboard. Folder upload, folder creation, and deletion were all tested to confirm the deployment is fully operational. A "Private Cloud" test folder was uploaded then deleted after confirming it appeared correctly (the folder contained a Claude API key).

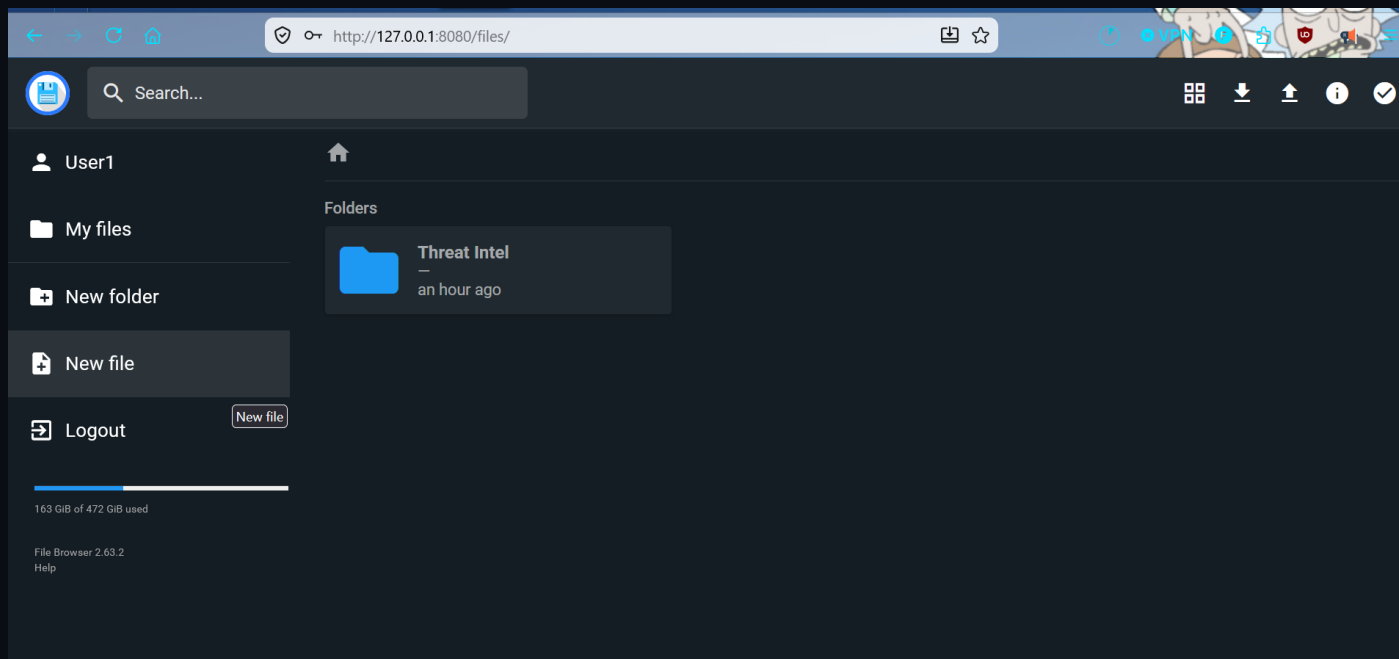


Fig 12. User1 dashboard showing the Threat Intel folder -- 163 GiB of 472 GiB used, File Browser v2.63.2

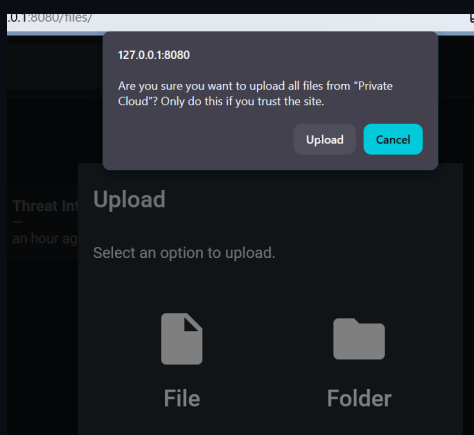


Fig 13. Upload modal -- browser prompts trust confirmation before u

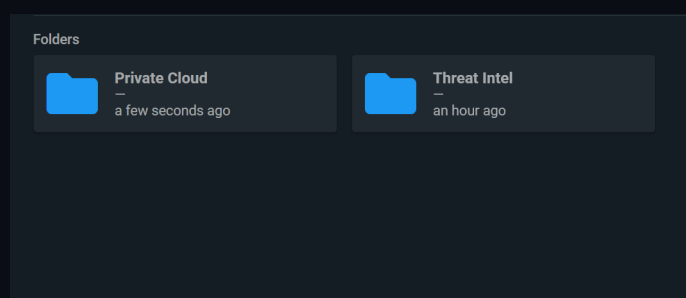


Fig 14. Both folders present after upload -- Private Cloud (just uploaded) + Threat

Folders

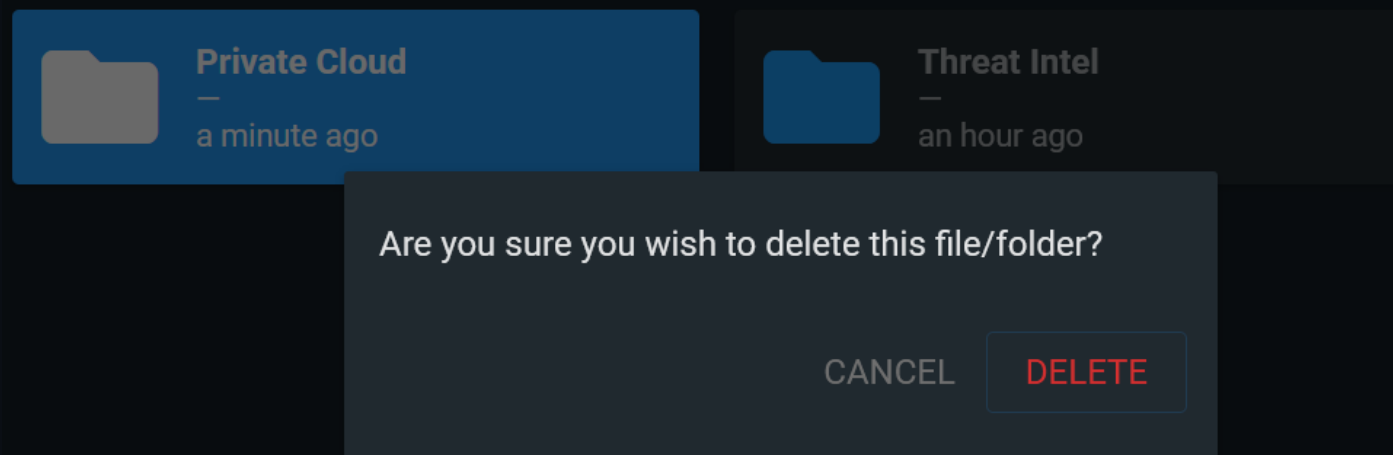


Fig 15. Delete confirmation dialog -- successfully removing the Private Cloud test folder

4

SECURITY & ACCESS CONTROLS

Credential Hygiene

Auto-generated password replaced on first login. Password changes require current credential re-entry.

Least Privilege

Admin account reserved for config only. User1 handles all routine file operations.

Localhost Binding

Service bound to 127.0.0.1 -- not reachable from LAN or internet without explicit firewall/proxy config.

Data Persistence

Files stored in C:\files on the host. Container destruction does not cause data loss.

Restart Policy

--restart unless-stopped provides availability without requiring host-level service installation.

Chunked Uploads

10MB chunk threshold with 5 retries provides reliable uploads even on unstable connections.